

Agile 방법론 및 MSA 개발 실습

온라인 교육 플랫폼 구축

Step 1. 요구사항 분석

목적: 전체 기능을 파악하고 서비스 경계 도출의 기반 마련

주체	기능
수강생	회원가입, 로그인, 강의 검색, 수강신청, 결제, 추천 강의 조회
강사	회원가입, 로그인, 강의 등록
시스템	결제 완료 후 수강 활성화, 수강 완료 후 추천 갱신

Step 2. 도메인 분리

목적: 기능을 묶어 서비스 경계 직접 도출

서비스	스택	포트	묶인 기능	핵심 데이터
User Service	Spring Boot	8081	회원가입, 로그인, JWT 인증	users
Course Service	Spring Boot	8082	강의 등록, 목록, 검색, 카테고리 관리	courses
Enrollment Service	Spring Boot	8083	수강신청, 수강 상태 관리	enrollments
Payment Service	Spring Boot	8084	결제 처리, 결제 내역	payments
Recommend Service	FastAPI	8085	수강 이력 기반 규칙 추천	enrollments + courses 조회

Step 3. 인프라 구성 설계

목적: 서비스를 둘러싼 인프라 요소와 역할 정의

인프라	포트	역할
Spring Cloud Gateway	8080	단일 진입점, 라우팅, 인증 필터
Eureka Server	8761	서비스 등록/탐색
Kafka Broker (KRaft)	9092	비동기 이벤트 메시지 버스
MariaDB	3306	단일 인스턴스, 테이블 단위 분리

Step 4. 서비스 간 통신 설계

목적: 동기/비동기 통신 구간 판단

동기 통신 (REST)

호출 방향	이유
Enrollment → Course	강의 존재 여부 즉시 확인 필요
Enrollment → Payment	결제 결과 즉시 필요
Recommend → Enrollment	수강 이력 조회
Recommend → Course	강의 목록/카테고리 조회

비동기 통신 (Kafka)

이벤트	Producer	Consumer	처리 내용
payment.completed	Payment Service	Enrollment Service	수강 상태 ACTIVE 로 변경
enrollment.completed	Enrollment Service	Recommend Service	추천 결과 갱신 트리거

Step 5. DB 설계 / ERD

목적: 테이블 구조 및 업무 선후 관계 설계

테이블 정의

테이블	컬럼	비고
users	id, name, email, password, role	role: STUDENT / INSTRUCTOR
courses	id, title, category, price, instructor_id	instructor_id → users.id
enrollments	id, user_id, course_id, status	status: PENDING / ACTIVE
payments	id, user_id, course_id, amount, status	status: COMPLETED / FAILED

Recommend Service DB 접근

- enrollments, courses 테이블 Read Only 조회
- 별도 테이블 불필요 (추론은 규칙 기반으로 진행)

Step 6. API 명세 설계

목적: 서비스 간 인터페이스 계약 정의

Gateway 라우팅

경로	대상 서비스
/users/**	User Service :8081
/courses/**	Course Service :8082
/enrollments/**	Enrollment Service :8083
/payments/**	Payment Service :8084
/recommend/**	Recommend Service :8085

User Service

Method	Endpoint	설명
POST	/users/register	회원가입
POST	/users/login	로그인 (JWT 발급)
GET	/users/{id}	회원 정보 조회

Course Service

Method	Endpoint	설명
POST	/courses	강의 등록 (강사)
GET	/courses	강의 목록 조회
GET	/courses/{id}	강의 상세 조회
GET	/courses/category/{category}	카테고리별 강의 조회

Enrollment Service

Method	Endpoint	설명
POST	/enrollments	수강 신청
GET	/enrollments/user/{userId}	수강 목록 조회
PATCH	/enrollments/{id}/status	수강 상태 변경 (Kafka 수신 후)

Payment Service

Method	Endpoint	설명
POST	/payments	결제 요청
GET	/payments/{id}	결제 내역 조회

Recommend Service (FastAPI)

Method	Endpoint	설명
GET	/recommend/{userId}	수강 이력 기반 추천 강의 목록 반환

추천 규칙 정의

1. 사용자의 수강 완료 강의 카테고리 추출
2. 동일 카테고리의 미수강 강의 목록 조회
3. 수강생 수 기준 내림차순 정렬 후 반환

Step 7. 컨테이너(Docker Compose) 구성 설계

목적: 전체 시스템을 컨테이너로 어떻게 묶을지 설계

컨테이너	이미지	포트	의존성
gateway	Spring Boot 빌드	8080	eureka
eureka	Spring Boot 빌드	8761	-
user-service	Spring Boot 빌드	8081	eureka, mariadb
course-service	Spring Boot 빌드	8082	eureka, mariadb
enrollment-service	Spring Boot 빌드	8083	eureka, mariadb, kafka
payment-service	Spring Boot 빌드	8084	eureka, mariadb, kafka
recommend-service	FastAPI 빌드	8085	eureka, mariadb
kafka	confluentinc/cp-kafka	9092	-
mariadb	mariadb:latest	3306	-

전체 프로젝트 구조

online-lecture-platform/

├── eureka-server/

├── auth-server/

├── api-gateway/

├── user-service/

└── course-service/

└── enrollment-service/
└── payment-service/
└── recommend-service/
└── docker-compose.yml

인증 흐름

Client → Auth Server (로그인 → Access Token 발급)
Client → Gateway → 각 서비스 (Access Token 첨부)
각 서비스 → Auth Server (토큰 검증, JWK Set 조회)

확정 기술 스택

항목	선택
Spring Boot	3.4.x
Java	21
빌드 도구	Gradle
OAuth2 Grant Type	Authorization Code (사용자) + Client Credentials (서비스 간)
서비스 간 REST 호출	WebClient
인증 구조	Spring Authorization Server + Resource Server
메시지 브로커	Kafka (KRaft)
DB	MariaDB
Service Discovery	Eureka
API Gateway	Spring Cloud Gateway

코드 작성 순서

1. eureka-server (서비스 등록/탐색 기반)
2. auth-server (인증 기반 먼저 확립)
3. api-gateway (라우팅 + 토큰 검증)
4. user-service (Resource Server 기본 패턴)
5. course-service (Resource Server)

6. enrollment-service (Resource Server + Kafka Producer/Consumer)
7. payment-service (Resource Server + Kafka Producer)
8. docker-compose.yml (전체 통합)

eureka-server 구성

eureka-server/

```

├── build.gradle
├── settings.gradle
├── gradle/wrapper/gradle-wrapper.properties
├── src/main/
│   ├── java/com/lecture/eureka/
│   │   └── EurekaServerApplication.java
│   └── resources/
│       └── application.yml

```

주요 설정 포인트

설정	값	이유
port	8761	Eureka 기본 포트
register-with-eureka	false	자기 자신 등록 불필요
fetch-registry	false	자기 자신 조회 불필요
enable-self-preservation	false	개발 환경에서 유령 서비스 방지

auth-server 구성

auth-server/

```

├── build.gradle

```

```

├── settings.gradle
├── src/main/java/com/lecture/auth/
│   ├── AuthServerApplication.java
│   ├── config/
│   │   ├── AuthorizationServerConfig.java ← 핵심
│   │   ├── UserDetailsConfig.java
│   │   └── DataInitializer.java
│   ├── model/
│   │   └── User.java
│   ├── repository/
│   │   └── UserRepository.java
│   └── service/
│       └── CustomUserDetailsService.java

```

주요 설계 포인트

항목	내용
포트	9000
Authorization Code	web-client → 사용자 로그인 처리
Client Credentials	service-client → 서비스 간 통신
JWT 서명	RSA 2048 키쌍 자동 생성
JWK Set URI	http://localhost:9000/oauth2/jwks (각 서비스가 공개키 조회)
초기 데이터	student / instructor 계정 자동 생성

주요 OAuth2 엔드포인트

POST /oauth2/token - 토큰 발급

GET /oauth2/authorize - 인증 코드 요청

GET /oauth2/jwks - 공개키 조회 (Resource Server 사용)

GET /.well-known/openid-configuration - OIDC 메타데이터

api-gateway 구성

api-gateway/

├── build.gradle

├── settings.gradle

└── src/main/java/com/lecture/gateway/

 ├── ApiGatewayApplication.java

 ├── config/

 └── SecurityConfig.java

 └── filter/

 ├── JwtAuthenticationFilter.java ← JWT 클레임 → 헤더 전달

 └── LoggingFilter.java ← 요청/응답 로깅

주요 설계 포인트

항목	내용
포트	8080
토큰 검증	Auth Server JWK Set URI 로 공개키 조회 후 검증
라우팅	Eureka lb:// 기반 로드밸런싱
헤더 전달	X-User-Id / X-User-Email / X-User-Role → 하위 서비스
공개 경로	/users/register, /users/login, /oauth2/**
필터 순서	LoggingFilter(-2) → JwtAuthenticationFilter(-1)

라우팅 요약

경로	대상 서비스
/users/**	user-service :8081
/courses/**	course-service :8082
/enrollments/**	enrollment-service :8083
/payments/**	payment-service :8084
/recommend/**	recommend-service :8085

user-service 구성

user-service/

```

├── build.gradle
├── settings.gradle
└── src/main/java/com/lecture/user/
    ├── UserServiceApplication.java
    ├── config/
    │   ├── SecurityConfig.java      ← Resource Server 설정
    │   ├── JpaConfig.java          ← Auditing 활성화
    │   └── GlobalExceptionHandler.java
    ├── controller/
    │   └── UserController.java
    ├── dto/
    │   └── UserDto.java             ← Register/Response/ApiResponse
    ├── entity/
    │   └── User.java
    └── repository/

```

```

|   └── UserRepository.java
└── service/
    └── UserService.java

```

주요 설계 포인트

항목	내용
포트	8081
인증	Resource Server, JWK Set URI 로 토큰 검증
/api/users/register	인증 불필요 (공개)
/api/users/internal/**	SCOPE_service.read 필요 (서비스 간 호출)
/api/users/me	JWT sub 클레임으로 본인 정보 조회
비밀번호	BCrypt 암호화
Auditing	createdAt / updatedAt 자동 관리

course-service 구성

course-service/

```

├── build.gradle
├── settings.gradle
└── src/main/java/com/lecture/course/
    ├── CourseServiceApplication.java
    ├── config/
    │   ├── SecurityConfig.java
    │   ├── JpaConfig.java
    │   └── GlobalExceptionHandler.java
    ├── controller/
    │   └── CourseController.java
    └── dto/

```

```

|   └── CourseDto.java
|
|── entity/
|
|   └── Course.java
|
|── repository/
|
|   └── CourseRepository.java
|
|── service/
|
|   └── CourseService.java

```

주요 설계 포인트

항목	내용
포트	8082
강의 등록	ROLE_INSTRUCTOR 만 가능
강의 조회	인증 불필요 (공개)
/internal/exists/{id}	Enrollment Service → 강의 존재 확인
/internal/{id}/enrollment-count	수강 활성화 시 수강생 수 증가
/internal/recommend	Recommend Service → 카테고리별 미수강 강의 조회
enrollmentCount	추천 정렬 기준, 수강 활성화 시 자동 증가

API 엔드포인트 요약

Method	Path	권한	설명
POST	/api/courses	INSTRUCTOR	강의 등록
GET	/api/courses	없음	전체 목록
GET	/api/courses/{id}	없음	상세 조회
GET	/api/courses/category/{category}	없음	카테고리별
GET	/api/courses/internal/exists/{id}	service.read	존재 확인
POST	/api/courses/internal/{id}/enrollment-count	service.read	수강생 수 증가
GET	/api/courses/internal/recommend	service.read	추천용 조회

enrollment-service 구성

enrollment-service/

├── build.gradle

└── src/main/java/com/lecture/enrollment/

├── config/

| ├── SecurityConfig.java

| ├── JpaConfig.java

| ├── KafkaConfig.java ← 토픽 자동 생성

| ├── WebClientConfig.java ← lb:// 로드밸런싱

| └── GlobalExceptionHandler.java

├── controller/

| └── EnrollmentController.java

├── dto/

| └── EnrollmentDto.java

├── entity/

| └── Enrollment.java

├── kafka/

| ├── KafkaEvent.java ← 이벤트 DTO

| ├── EnrollmentKafkaConsumer.java ← payment.completed 수신

| └── EnrollmentKafkaProducer.java ← enrollment.completed 발행

├── repository/

| └── EnrollmentRepository.java

└── service/

├── EnrollmentService.java ← 핵심 비즈니스 로직

└── CourseServiceClient.java ← WebClient
└── PaymentServiceClient.java ← WebClient

핵심 흐름

수강신청 요청

- Course Service REST (강의 존재 확인)
- Enrollment 생성 (PENDING)
- Payment Service REST (결제 요청)

[Kafka Consumer]

payment.completed 수신

- Enrollment ACTIVE 전환
- Course Service REST (수강생 수 증가)
- enrollment.completed 발행 (→ Recommend Service)

API 엔드포인트

Method	Path	권한	설명
POST	/api/enrollments	인증 필요	수강신청
GET	/api/enrollments/user/{userId}	인증 필요	수강 목록
GET	/api/enrollments/internal/history/{userId}	service.read	추천용 이력 조회

payment-service 구성

payment/

└── PaymentServiceApplication.java
└── config/
| └── SecurityConfig.java
| └── JpaConfig.java
| └── GlobalExceptionHandler.java

```

|—— controller/
|   |—— PaymentController.java
|
|—— dto/
|   |—— PaymentDto.java
|
|—— entity/
|   |—— Payment.java
|
|—— kafka/
|   |—— PaymentKafkaProducer.java
|
|—— repository/
|   |—— PaymentRepository.java
|
|—— service/
|   |—— PaymentService.java

```

핵심 흐름

Enrollment Service → POST /payments/internal/request

- Payment 생성 (PENDING)
- 결제 처리 (실습: UUID 트랜잭션 ID)
- Payment 상태 COMPLETED
- Kafka: payment.completed 발행
- Enrollment Service (수강 활성화)

API 엔드포인트

Method	Path	권한	설명
POST	/api/payments/internal/request	service.read	내부 결제 요청
GET	/api/payments/{id}	인증 필요	결제 단건 조회
GET	/api/payments/user/{userId}	인증 필요	결제 내역 조회

공통 Build Gradle

```
plugins {  
    id 'java'  
  
    id 'org.springframework.boot' version '3.4.1'  
  
    id 'io.spring.dependency-management' version '1.1.7'  
}
```

```
group = 'com.lecture'  
  
version = '0.0.1-SNAPSHOT'
```

```
java {  
    toolchain {  
        languageVersion = JavaLanguageVersion.of(21)  
    }  
}
```

```
repositories {  
    mavenCentral()  
}
```

```
ext {  
    set('springCloudVersion', "2024.0.0")  
}
```

```
dependencies {
```

```

        implementation 'org.springframework.cloud:spring-cloud-starter-netflix-
eureka-server'

        testImplementation 'org.springframework.boot:spring-boot-starter-test'
    }

    dependencyManagement {

        imports {

            mavenBom "org.springframework.cloud:spring-cloud-
dependencies:${springCloudVersion}"

        }

    }

    tasks.named('test') {

        useJUnitPlatform()

    }

```

recommend-service 구성

recommend-service/

```

├── main.py                                ← FastAPI 앱 진입점
├── requirements.txt
├── Dockerfile
├── .env
├── app/
    └── config/

```



```

|   |—— settings.py           ← 환경 설정
|   |—— security.py           ← JWT 검증
|   |—— model/
|   |—— schemas.py            ← Pydantic 스키마
|   |—— client/
|   |—— enrollment_client.py   ← Enrollment Service 호출
|   |—— course_client.py      ← Course Service 호출
|   |—— service/
|   |—— recommend_service.py   ← 추천 핵심 로직
|   |—— kafka/
|   |—— consumer.py           ← enrollment.completed 수신
|   |—— router/
|   |—— recommend_router.py    ← API 엔드포인트

```

추천 규칙 흐름

GET /recommend/{userId}

- Enrollment Service: 수강 이력(activeCourseIds) 조회
- Course Service: 전체 강의 목록으로 카테고리 분석
- Counter 로 최빈 카테고리 선택
- Course Service: 해당 카테고리 미수강 강의 조회
- 수강생 수 기준 정렬 → 상위 5 개 반환

수강 이력 없는 신규 사용자:

- 전체 강의 수강생 수 기준 인기순 상위 5 개 반환

docker-compose.yml 구성

전체 컨테이너 구성 (9 개)

컨테이너	이미지	포트	의존성
mariadb	mariadb:11.2	3306	-

kafka	confluentinc/cp-kafka:7.7.0	9092	- (KRaft)
eureka-server	빌드	8761	-
auth-server	빌드	9000	mariadb, eureka
api-gateway	빌드	8080	eureka, auth
user-service	빌드	8081	mariadb, eureka, auth
course-service	빌드	8082	mariadb, eureka, auth
enrollment-service	빌드	8083	mariadb, eureka, auth, kafka
payment-service	빌드	8084	mariadb, eureka, auth, kafka
recommend-service	빌드	8085	eureka, auth, kafka

기동 순서 (depends_on 기반)

MariaDB / Kafka (인프라)

→ Eureka (서비스 등록)

→ Auth Server (인증)

→ API Gateway + 4 개 서비스

→ Recommend Service

전체 기동 명령

bash

프로젝트 루트에서 (초기 트러블슈팅/리빌드 고려, 캐시 없이 빌드, 컨테이너는 묶어서 백그라운드로 실행)

docker compose build --no-cache

docker compose up -d

또는 한줄로

docker compose build --no-cache && docker compose up -d

```
phoenix@Phoenix-MacBook-Pro 2026_MSA % docker compose build --no-cache && docker compose up -d
=> [payment-service] resolving provenance for metadata file 0.0s
=> [enrollment-service stage-1 3/3] COPY --from=builder /app/build/libs/*.jar app.jar 0.3s
=> [enrollment-service] exporting to image 0.2s
=> => exporting layers 0.2s
=> => writing image sha256:8783b07e03cf8070ad278c81c893778d4e5cb7c641c68c507b58d10ab438fa01 0.0s
=> => naming to docker.io/library/2026_msa-enrollment-service 0.0s
=> [enrollment-service] resolving provenance for metadata file 0.0s
[+] build 8/8
✓ Image 2026_msa-recommend-service Built 151.5s
✓ Image 2026_msa-enrollment-service Built 151.5s
✓ Image 2026_msa-auth-server Built 151.5s
✓ Image 2026_msa-api-gateway Built 151.5s
✓ Image 2026_msa-user-service Built 151.5s
✓ Image 2026_msa-payment-service Built 151.5s
✓ Image 2026_msa-eureka-server Built 151.5s
✓ Image 2026_msa-course-service Built 151.5s
[+] up 11/11
✓ Network 2026_msa_lecture-net Created 0.0ss
✓ Container lecturedb Healthy 11.8s
✓ Container lecture-eureka Healthy 17.7s
✓ Container lecture-kafka Healthy 17.7s
✓ Container lecture-auth Healthy 17.5s
✓ Container lecture-course Created 0.1ss
✓ Container lecture-payment Created 0.1ss
✓ Container lecture-enrollment Created 0.1ss
✓ Container lecture-user Created 0.1ss
✓ Container lecture-gateway Created 0.1ss
✓ Container lecture-recommend Created 0.0ss
○ phoenix@Phoenix-MacBook-Pro 2026_MSA %
```

로그 확인

전체 로그 한번에 보기

docker compose logs -f

또는 개별 컨테이너 로그 보기

docker compose logs -f [서비스명]

docker compose logs -f mariadb

docker compose logs -f kafka

docker compose logs -f eureka-server

docker compose logs -f auth-server

docker compose logs -f api-gateway

docker compose logs -f user-service

docker compose logs -f course-service

docker compose logs -f enrollment-service

docker compose logs -f payment-service

docker compose logs -f recommend-service

전체 종료 (또는 컨테이너 빌드 중, 실패 시에 기존 컨테이너 정리)

docker compose down

서버 기동 상태 확인

<http://localhost:8761/>

The screenshot shows the Spring Eureka dashboard. At the top, there's a navigation bar with the Spring Eureka logo and links for HOME and LAST 1000 SINCE STARTUP. The main content area is divided into several sections:

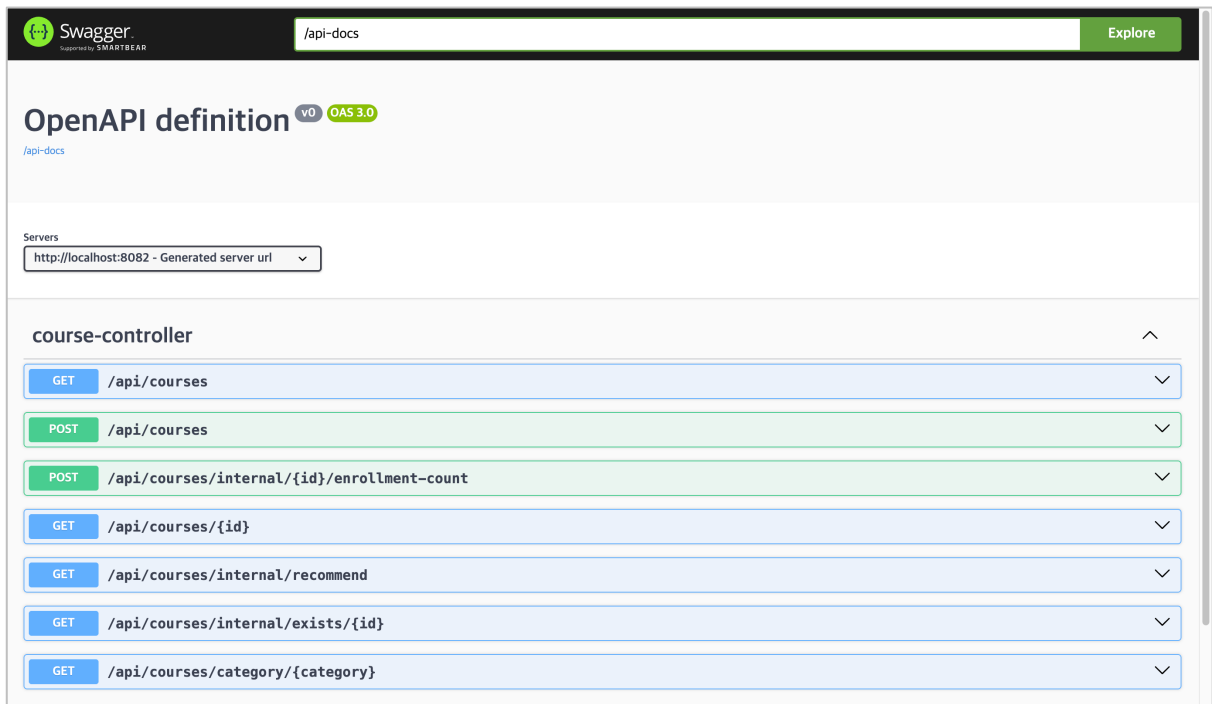
- System Status:** A table showing environment (test), data center (default), current time (2026-03-12T00:17:11 +0000), uptime (02:37), lease expiration enabled (true), renewals threshold (0), and renewals (last min) (14).
- THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.** A red warning message.
- DS Replicas:** A section for distributed replicas.
- Instances currently registered with Eureka:** A table listing various services and their status.
- General Info:** A section for general information.

Application	AMIs	Availability Zones	Status
API-GATEWAY	n/a (1)	(1)	UP (1) - 436d7852cc8d:api-gateway:8080
AUTH-SERVER	n/a (1)	(1)	UP (1) - 6314b7349e62:auth-server:9000
COURSE-SERVICE	n/a (1)	(1)	UP (1) - 9f1238855c26:course-service:8082
ENROLLMENT-SERVICE	n/a (1)	(1)	UP (1) - 5646df269da6:enrollment-service:8083
PAYMENT-SERVICE	n/a (1)	(1)	UP (1) - 31e1448330a5:payment-service:8084
RECOMMEND-SERVICE	n/a (1)	(1)	UP (1) - 172.19.0.11:recommend-service:8085
USER-SERVICE	n/a (1)	(1)	UP (1) - 3f4296d33b18:user-service:8081

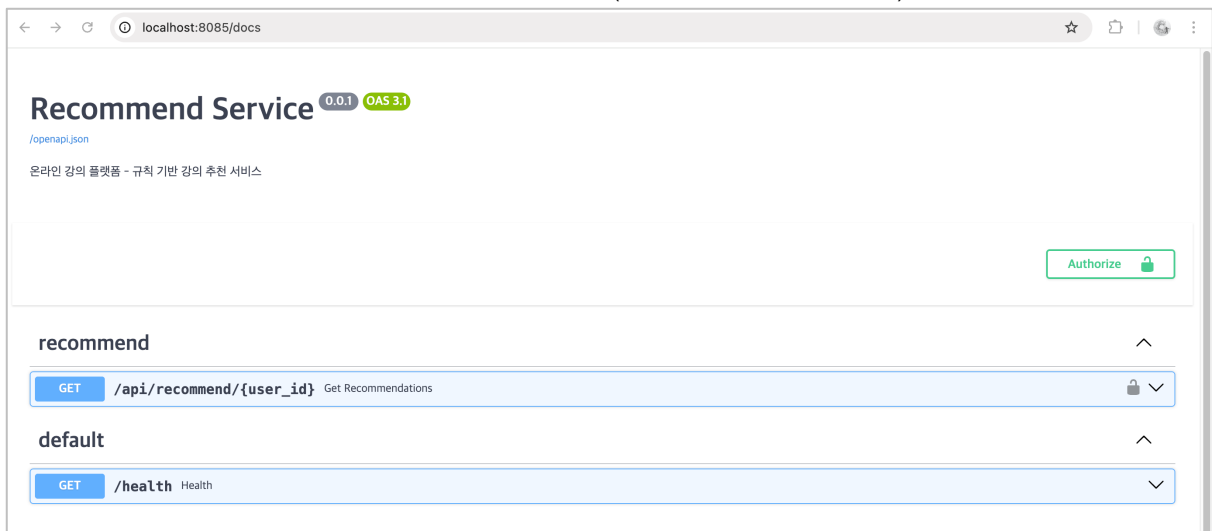
Swagger UI 로 REST API 엔드포인트 확인

서비스	Swagger UI URL
user-service	http://localhost:8081/swagger-ui.html
course-service	http://localhost:8082/swagger-ui.html
enrollment-service	http://localhost:8083/swagger-ui.html
payment-service	http://localhost:8084/swagger-ui.html
recommend-service	http://localhost:8085/docs

course-service 엔드포인트 예시 (REST API-Java)



recommend-service 엔드포인트 예시 (REST API-FastAPI)



전체 워크플로우 다이어그램

flowchart TB

Client(["👤 Client (Browser/App)"])

```
subgraph Gateway["API Gateway Layer"]
    GW["Spring Cloud GatewayWn:8080"]
end
```

```
subgraph Discovery["Service Discovery"]
    EU["Eureka ServerWn:8761"]
end
```

SKALA 임성열 교수.

Agile 방법론 및 MSA 개발

```

end

subgraph Services["Microservices"]
    US["User ServiceWn:8081WnSpring Boot"]
    CS["Course ServiceWn:8082WnSpring Boot"]
    ES["Enrollment ServiceWn:8083WnSpring Boot"]
    PS["Payment ServiceWn:8084WnSpring Boot"]
    RS["Recommend ServiceWn:8085WnFastAPI"]
end

subgraph MariaDB["MariaDB (Single Instance)"]
    TB1[("usersWnid, name, email, role")]
    TB2[("coursesWnid, title, category, instructor_id, price")]
    TB3[("enrollmentsWnid, user_id, course_id, status")]
    TB4[("paymentsWnid, user_id, course_id, amount, status")]

    TB1 -->|"강사/수강생 식별"| TB2
    TB1 -->|"수강 신청자"| TB3
    TB2 -->|"수강 대상 강의"| TB3
    TB3 -->|"수강 확정 후 결제"| TB4
    TB3 -->|"수강 이력 기반 추천"| TB2
end

subgraph Kafka["Async Message Bus (KRaft)"]
    KF["Kafka BrokerWn:9092"]
end

subgraph Docker["Docker Compose"]
    direction LR
    D1["gateway container"]
    D2["eureka container"]
    D3["springbootWncontainers"]
    D4["fastapi container"]
    D5["kafka container"]
    D6["mariadb container"]
end

Client -->|"HTTP Request"| GW

GW -->|"Route /users/**"| US
GW -->|"Route /courses/**"| CS
GW -->|"Route /enrollments/**"| ES
GW -->|"Route /payments/**"| PS
GW -->|"Route /recommend/**"| RS

US & CS & ES & PS & RS -->|"Register/Heartbeat"| EU
GW -->|"Lookup"| EU

ES -->|"REST: 강의 존재 확인"| CS
ES -->|"REST: 결제 요청"| PS

```

```
RS -->|"REST: 수강 이력 조회"| ES
RS -->|"REST: 강의 목록 조회"| CS
```

```
PS -->|"payment.completed"| KF
KF -->|"수강 활성화"| ES
```

```
ES -->|"enrollment.completed"| KF
KF -->|"추천 갱신 트리거"| RS
```

```
US --- TB1
CS --- TB2
ES --- TB3
PS --- TB4
RS -.|"Read Only"wn 수강이력/강의목록"| TB2
RS -.|"Read Only"wn 수강이력/강의목록"| TB3
```

```
GW -.->|container| D1
EU -.->|container| D2
US & CS & ES & PS -.->|container| D3
RS -.->|container| D4
KF -.->|container| D5
MariaDB -.->|container| D6
```

```
classDef service fill:#4A90D9,stroke:#2C5F8A,color:#fff
classDef fastapi fill:#009688,stroke:#00695C,color:#fff
classDef table fill:#F5A623,stroke:#C47D0E,color:#fff
classDef infra fill:#7ED321,stroke:#5A9A18,color:#fff
classDef kafka fill:#D0021B,stroke:#9B0015,color:#fff
```

```
class US,CS,ES,PS service
class RS fastapi
class TB1,TB2,TB3,TB4 table
class GW,EU infra
class KF kafka
```

수강신청 시퀀스 다이어그램

sequenceDiagram

actor Client

participant GW as Gateway

participant ES as Enrollment Service

participant CS as Course Service

participant PS as Payment Service

participant KF as Kafka

participant RS as Recommend Service

Client->>GW: POST /enrollments

GW->>ES: 라우팅

ES->>CS: REST: 강의 존재 확인

CS-->>ES: 강의 정보 반환

ES->>PS: REST: 결제 요청

PS-->>ES: 결제 완료 응답

PS->>KF: payment.completed 발행

KF->>ES: 수강 상태 ACTIVE 변경

KF->>RS: 추천 갱신 트리거

RS->>CS: REST: 강의 목록 조회

RS->>ES: REST: 수강 이력 조회

RS-->>RS: 규칙 기반 추천 갱신

ES-->>GW: 수강신청 완료 응답

GW-->>Client: 200 OK

ERD 다이어그램

erDiagram

users {

bigint id PK

varchar name

varchar email

varchar password

varchar role

}

courses {


```
    bigint id PK
    varchar title
    varchar category
    decimal price
    bigint instructor_id FK
}
```

```
enrollments {
    bigint id PK
    bigint user_id FK
    bigint course_id FK
    varchar status
}
```

```
payments {
    bigint id PK
    bigint user_id FK
    bigint course_id FK
    decimal amount
    varchar status
}
```

users ||--o{ courses : "강사가 강의 개설"

users ||--o{ enrollments : "수강생이 수강 신청"

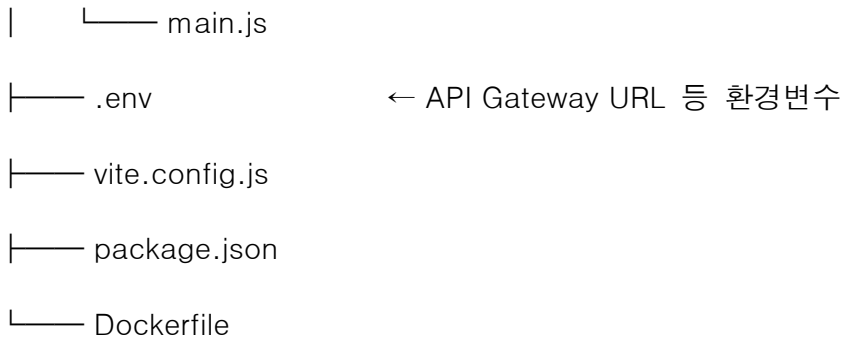
courses ||--o{ enrollments : "강의가 수강 대상"

enrollments ||--|| payments : "수강 확정 후 결제"

프론트엔드 개발 (Vue.js)

vue-frontend/

- |— src/
 - | |— assets/ ← 이미지, 글로벌 CSS
 - | |— components/ ← 공통 컴포넌트 (Header, Nav, CourseCard 등)
 - | |— views/ ← 페이지 단위
 - | | |— LoginView.vue
 - | | |— CourseListView.vue
 - | | |— CourseDetailView.vue
 - | | |— EnrollmentView.vue
 - | | |— MyPageView.vue
 - | |— store/ ← Pinia 상태 관리
 - | | |— auth.js ← 토큰, 사용자 정보
 - | | |— course.js ← 강의 목록 캐싱
 - | |— api/
 - | | |— index.js ← axios 인터셉터 설정
 - | | |— auth.js ← 로그인/토큰 API
 - | | |— course.js ← 강의 API
 - | | |— enrollment.js ← 수강신청 API
 - | |— router/
 - | | |— index.js ← 라우팅 + 인증 가드
 - | |— App.vue



적용 기술 스택

항목	선택
프레임워크	Vue 3
빌드 도구	Vite
상태 관리	Pinia
HTTP 클라이언트	axios
라우터	Vue Router 4
배포	nginx Docker 컨테이너

전체 구조 요약

파일	역할
package.json	Vue3 + Vite + Pinia + axios + vue-router
vite.config.js	@ alias, dev proxy → localhost:8080
.env	API Gateway URL, OAuth2 설정
src/main.js	앱 진입점
src/router/index.js	라우팅 + 인증 가드
src/store/auth.js	토큰/사용자 상태, OAuth2 흐름
src/store/course.js	강의 목록 캐싱, 썸네일 매핑
src/api/*.js	axios 인터셉터 + 각 서비스 API
src/components/AppHeader.vue	공통 헤더 (로고 + nav + 사용자)
src/components/CourseCard.vue	강의 카드 컴포넌트
src/views/*.vue	Landing, Login, CourseList, CourseDetail, Enrollment, MyPage, Callback
Dockerfile + nginx.conf	빌드 + 서빙

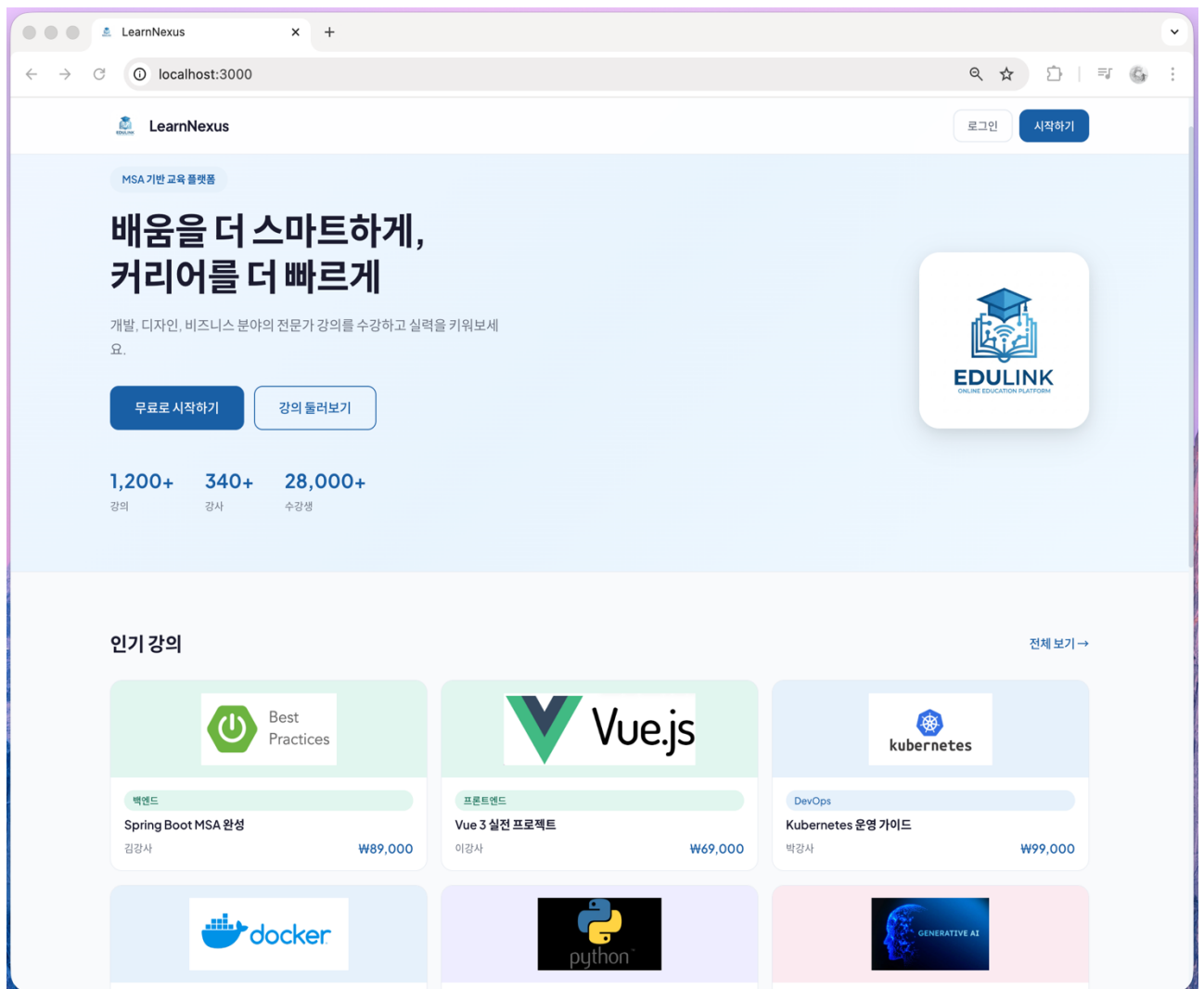
로컬 실행 방법

cd vue-frontend

npm install

npm run dev

브라우저에서 `http://localhost:3000` 접속



[트러블슈팅 가이드]

현재 진행 중인, API Gateway + Authorization Server + MSA 구조에서 인증 연동 트러블슈팅은 난이도가 있는 예제로 집중 필요

- Authorization Server 구성
- OAuth2 Authorization Code Flow 동작
- Gateway JWT validation
- JWK endpoint 연동
- issuer 검증 동작
- downstream header propagation
- user-service 인증 연동 완료

단일 서버 (Spring Boot 단일 앱)에서는 인증 흐름이 단순

client → server

그러나 현재 인증 구조는 다음과 같음

client → api-gateway → auth-server → user-service → course-service →
payment-service

이 과정에서 다음 요소들이 모두 맞아야 정상 동작함

- issuer → JWT 를 발급한 인증 서버의 식별자(URL)
- jwk-set-uri → JWT 서명 검증용 공개키(public key) 목록 제공 주소
- redirect-uri → 로그인 완료 후 인증 서버가 사용자를 되돌려보내는 주소
- client-id → OAuth2 인증 요청을 보내는 클라이언트 애플리케이션 식별자
- client-secret → client-id 에 대응하는 클라이언트 인증용 비밀번호
- scope → 클라이언트 또는 사용자가 접근 가능한 권한 범위(permission 범주)
- grant type → 토큰을 발급받는 인증 방식 종류
- JWT claim 구조 → JWT 내부에 저장되는 사용자 정보 및 인증 메타데이터

- **gateway header 전달 방식** → gateway 가 JWT 검증 후 downstream service 로 사용자 정보를 전달하는 방법
- **downstream service 인증 방식** → 내부 서비스(user-service 등)가 사용자 인증 정보를 처리하는 방식

인증은 단순한 기능이 아니라 **시스템 전체 계약(contract)**이며 다음이 모두 연결:

- 네트워크
- 토큰 구조
- 필터 체인
- 시큐리티 설정
- 서비스 간 통신 방식
- 프론트 redirect 흐름
- OAuth2 spec

일반 CRUD API 보다 상대적으로 다루는 기술 요소가 많으므로, 의미 있는 Web 서비스 개발 트러블슈팅 경험으로 정리할 것